

SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT2 - VM Configuration and Security Evidence Workbook

Guided practical workbook for Azure VM, local VM, Docker container, security checks, recovery evidence and GitHub submission



Assessment Metadata

Course Code	CSA15
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT2 - VM Configuration and Security Evidence
Submission Type	Individual digital workbook based on the same capstone title used in CO1 and CO2 AT1
Faculty	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Submission Mode	Completed workbook, evidence screenshots and GitHub repository link through Google Classroom

How This Workbook Works

- Use the same capstone title carried from CO1 and CO2 AT1.
- Read each procedure block, perform the action, verify the checkpoint and then fill the response table.
- Use Azure first when account activation is available. Use the local VM path when Azure is blocked. Use Docker for container comparison. Use simulation only when practical deployment is blocked.
- Screenshots must show the student's actual configuration or verified simulation work. Hide credentials, keys, tokens and sensitive account details.
- Upload the completed workbook and evidence to GitHub, then submit the GitHub repository link through Google Classroom.

Virtualization Tracks for CO2 AT2

Azure is the preferred cloud path. Local VM and container tracks are included for hands-on resilience.

Track A - Azure Linux VM

Preferred cloud evidence:
subscription, VM, SSH, NGINX/test
service, firewall, shutdown

Track B - Local VM

VirtualBox or VMware with Tiny
Core, Alpine, Debian netinst or
Ubuntu Server

Track C - Docker / Podman

Run a lightweight service, map a
port, inspect logs, compare with
VM

Track D - Simulation

Used only if deployment is
blocked; must include technical
configuration and security
evidence

Required common output: completed workbook, screenshots, command outputs, security choices, README.md, GitHub repository
link and GCR submission.

Technical Orientation

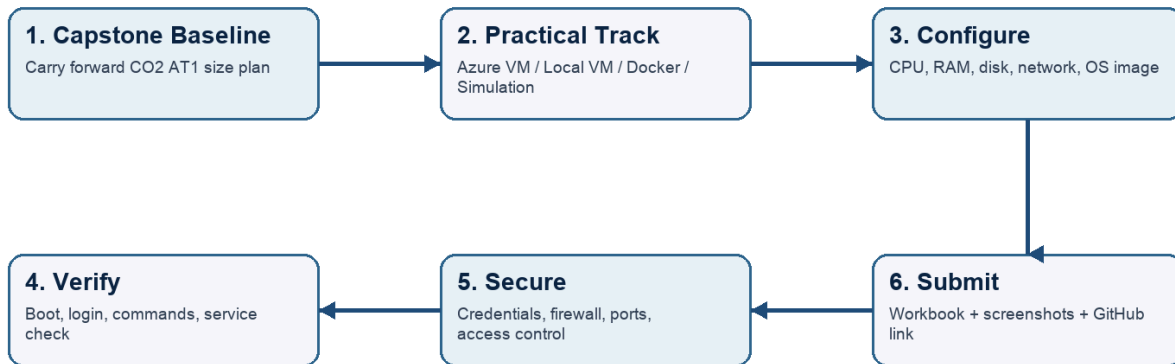
The practical work is not limited to clicking through screens. Each checkpoint requires observation: what appeared on screen, why it appeared, what it proves and how it connects to the capstone infrastructure plan.

Part 1: Student and Capstone Baseline

Student Name	Mukilan R
Register Number	192421043
Class / Slot	Slot B
Capstone Title	SkillForge-CO-Attainment Cloud Tutor Using Recommendation Analytics
CO2 AT1 Recommended VM Size	vCPU: 8 RAM: 32 GB Disk: 100 GB SSD
Selected Practical Track	Azure VM (in progress) and Local VM (completed); Docker attempted, troubleshooting in progress
GitHub Repository Link	To be added after GitHub repository is created and evidence uploaded
Google Classroom Submission Date	29/07/2026

CO2 AT2 Practical Evidence Flow

Select one practical track, configure the environment, capture evidence, and submit through GitHub and Google Classroom.



Capstone Infrastructure Element	Carry-Forward from CO2 AT1
Workload type	AI/Python module (primary - recommendation engine), with Analytics app and Database-heavy app as secondary characteristics (carried from CO2 AT1)
Main application modules	Login/Auth, Student Dashboard, Quiz/Assessment, CO-Attainment Tracker, Recommendation Engine, Faculty Upload Portal, Analytics Dashboard
Expected users / traffic assumption	1000 registered users; ~10% daily active (~250 DAU); peak factor 2 gives ~10 RPS at peak (from CO2 AT1 traffic calibration)
Database need	Yes - student records, CO mapping and quiz scores; planned as a managed database service
File storage need	Yes - course materials and assignment PDFs; planned as object storage, not VM disk
AI/Python/analytics need	Yes - recommendation engine is the core differentiating feature of the capstone
Security concern	Protecting student CO-attainment records and credentials; keeping inbound ports minimal and hiding all secrets from evidence screenshots
Recovery concern	Snapshot before deployment/demo/update; managed database backup for the production database tier

Part 2: OS Image Selection Guide

Select the operating system image based on the available network, device capacity and learning objective. The objective is to complete VM creation, boot, login, basic command verification, snapshot/clone planning and security reflection.

OS Option	Approximate Download Size / Nature	Best Use in This Assessment	Link
-----------	------------------------------------	-----------------------------	------

Tiny Core Linux	TinyCore around 23 MB; very lightweight GUI option.	Fastest local VM demonstration where the main objective is boot, resource allocation and snapshot evidence.	https://distro.ibiblio.org/tinycorelinux/downloads.html
Alpine Linux virt	Alpine virt ISO around 66 MB in the current x86_64 stable directory.	Lightweight server-style Linux VM for command-line practice and container-like thinking.	https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/
Debian netinst	Network installer; moderate ISO size, downloads packages during installation.	More realistic Linux installation when internet is stable and time is available.	https://www.debian.org/download
Ubuntu Server	Larger server ISO; familiar cloud VM image.	Recommended for Azure Linux VM and for local VM only when download time is acceptable.	https://ubuntu.com/download/server

Selection Guidance

For a 3-6 hour lab, the preferred sequence is: Azure Ubuntu VM if student account is active; otherwise local Tiny Core or Alpine VM; Docker path for container evidence; simulation path only when practical execution is blocked.

Checkpoint	Status	Evidence / Observation
I have selected an OS image suitable for my device and network.	[X] Yes [] Rework	Ubuntu Server LTS selected for the planned Azure VM; Tiny Core Linux selected for the local VirtualBox VM - both fit lab device and network constraints
I have recorded the official download page or cloud image name.	[X] Yes [] Rework	Tiny Core Linux: distro.ibiblio.org/tinycorelinux/ ; Azure image: Ubuntu Server 24.04 LTS (Azure Marketplace)
I have checked whether the ISO is small enough for the lab duration.	[X] Yes [] Rework	Tiny Core ISO (~23 MB) confirmed lightweight; Azure Ubuntu image is a managed cloud image so no local download is needed

Student Entry: Selected OS image, version, reason and link used

Part 3: Track A - Azure Linux VM Guided Procedure

Preferred Cloud Path

Use Azure first when the student account is active. Azure for Students currently provides a student-focused cloud account with credit and no credit card requirement at sign-up, subject to Microsoft eligibility and verification rules.

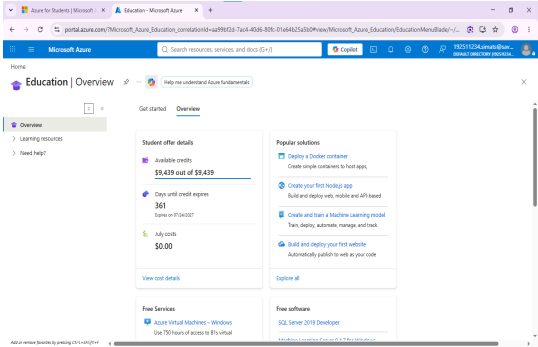
Azure for Students: <https://azure.microsoft.com/en-us/free/students>
Azure for Students offer details: <https://azure.microsoft.com/en-us/pricing/offers/ms-azr-0170p>
Azure Linux VM quickstart: <https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal>

A. Account Activation and Portal Access

1. Open the Azure for Students page and choose the student sign-up option.
2. Sign in using the official student email account when available.
3. Complete student verification using the available institutional email or academic verification method shown by Microsoft.
4. Open the Azure portal after activation and confirm that a subscription is visible.
5. Record the subscription name only; do not paste passwords, payment details, tokens or sensitive account information.

Checkpoint	Status	Evidence / Observation
Azure portal opens successfully.	[X] Yes [] Rework	Azure for Students overview page confirmed account active with full credit balance (\$9,439) available
A student or valid Azure subscription is visible.	[X] Yes [] Rework	Azure for Students subscription visible and active in the portal
The student understands that resources must be stopped or deleted after evidence capture.	[X] Yes [] Rework	Understood - the VM will be stopped/deallocated or deleted after evidence capture to protect the student credit

Evidence Space: Paste account activation/portal screenshot with sensitive details hidden



B. Create the Linux Virtual Machine

6. In Azure portal, search for Virtual machines and choose Create.
7. Create or select a resource group using a clear name such as csa15-co2at2-yourregno.
8. Enter a VM name such as csa15-vm-yourregno.
9. Select a nearby Azure region available in the subscription.
10. Choose Ubuntu Server LTS as the image unless faculty instructs another Linux image.
11. Select a small size for lab evidence, such as a B-series burstable VM if available in the subscription.
12. Select SSH public key or password according to faculty guidance. Do not share credentials in the workbook.
13. Allow SSH port 22 only when required. If a web service is tested, add HTTP port 80 only for the test window.
14. Review and create the VM. Wait until deployment succeeds.

Azure Setting	Student Value
Resource group	csa15-co2at2-192511234 (planned)
VM name	csa15-vm-192511234 (planned)
Region	Pending deployment - nearest available region in subscription
Image	Ubuntu Server 24.04 LTS (planned)
VM size	B-series burstable, e.g. B1s (planned)
vCPU	Pending deployment
RAM	Pending deployment
Disk type and size	Pending deployment
Authentication method	SSH key / password - do not paste secret
Open inbound ports	SSH (22) only - planned

Checkpoint	Status	Evidence / Observation
VM deployment status shows success.	☐ Yes ☒ Rework	VM not yet deployed - deployment step pending at time of this workbook entry
VM overview page shows running status.	☐ Yes ☒ Rework	Pending VM creation
Public IP or connection method is visible.	☐ Yes ☒ Rework	Pending VM creation
The selected VM size is compared with CO2 AT1 sizing recommendation.	☐ Yes ☒ Rework	To be completed once deployed - planned burstable size is intentionally much smaller than the CO2 AT1 production estimate (8 vCPU/32GB) since this is lab evidence, not production scale
Evidence Space: Paste VM overview and configuration screenshot		

C. Connect and Verify the VM

15. Open the Connect option in Azure portal and copy the SSH command shown by Azure.
16. Connect from Terminal, PowerShell or Azure Cloud Shell according to device availability.
17. After login, run: `uname -a`
18. Run: `lsb_release -a` or `cat /etc/os-release`
19. Run: `free -h`
20. Run: `df -h`
21. Run: `ip addr` or `hostname -l`
22. Record the output as screenshot evidence.

Verification Command	Expected Purpose	Student Observation
<code>uname -a</code>	Kernel and OS environment	Pending - VM not yet deployed
<code>cat /etc/os-release</code>	Linux distribution details	Pending - VM not yet deployed
<code>free -h</code>	RAM allocation	Pending - VM not yet deployed

df -h	Disk allocation	Pending - VM not yet deployed
hostname -I	Private IP / network evidence	Pending - VM not yet deployed
Evidence Space: Paste command output screenshot or copied terminal output		

D. Optional Web Service Check

23. Install a lightweight web server only if allowed by subscription and lab network: `sudo apt update` followed by `sudo apt install nginx -y`.
24. Start or verify the service: `systemctl status nginx`.
25. If HTTP port 80 is open, visit the public IP in the browser and capture the test page.
26. If port 80 is not opened, record the reason and keep SSH-only evidence.

Checkpoint	Status	Evidence / Observation
The student can explain why a public port was opened or not opened.	[] Yes [X] Rework	Not reached yet - Azure VM not deployed
The service status or browser test was captured.	[] Yes [X] Rework	Not reached yet
The public exposure was kept temporary for the assessment.	[] Yes [X] Rework	N/A - no service deployed yet

E. Azure Security and Cost-Control Closeout

27. Review Networking and record inbound port rules.
28. Confirm that no unnecessary public ports are open.
29. Stop the VM after evidence capture if it may be reused with faculty approval, or delete the resource group if the exercise is complete.
30. Capture stopped/deallocated or deletion evidence.
31. Record what would change for a production capstone deployment.

Security / Cost-Control Item	Student Evidence / Decision
Inbound ports allowed	Planned: SSH (22) only
Authentication method used	Planned: SSH key
Public IP required?	Planned: yes, temporarily, for SSH access only
Stopped/deallocated or deleted?	Not yet deployed
Reason for closeout choice	N/A - pending deployment
Evidence Space: Paste networking rule and shutdown/deletion screenshot	

Part 4: Track B - Local VM Guided Procedure

Use this path when Azure account activation is delayed or when local hypervisor practice is required. VirtualBox is recommended for broad accessibility. VMware Workstation/Fusion may be used where available.

VirtualBox downloads: <https://www.virtualbox.org/wiki/Downloads>

VMware Workstation and Fusion: <https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion>

A. Install the Hypervisor

32. Download VirtualBox or VMware only from the official website.
33. Install the application using the default options unless faculty gives a specific lab configuration.
34. Restart the system if the installer requests it.
35. Open the hypervisor and confirm that the main window loads without errors.
36. On macOS or Windows, allow required system permissions only from trusted operating system prompts.

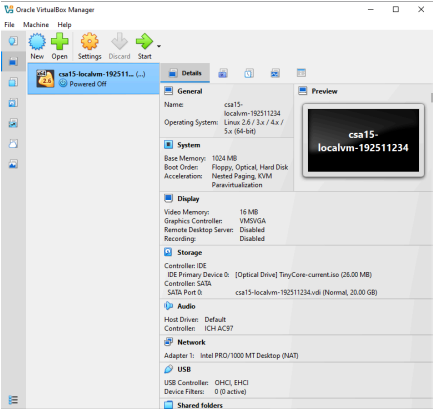
Item	Student Entry
Hypervisor selected	VirtualBox
Version	Not captured from screenshot - to confirm via Help > About VirtualBox
Host operating system	Windows
Host CPU and RAM	Not captured - to confirm via System Information
Reason for selected hypervisor	VirtualBox is free, widely available and sufficient for lightweight Linux VM lab evidence

Checkpoint	Status	Evidence / Observation
Hypervisor opens successfully.	[X] Yes [] Rework	VirtualBox opened and the VM window is running (Tiny Core boot screen visible)
Student can locate New VM/Create VM option.	[X] Yes [] Rework	VM created and named csa15-localvm-192511234
Student has downloaded or selected a suitable OS image.	[X] Yes [] Rework	Tiny Core Linux selected as the lightweight OS image

B. Create the VM

37. Choose New VM or Create a New Virtual Machine.
38. Enter a clear VM name such as csa15-localvm-yourregno.
39. Select Linux as the type and the closest version available.
40. Attach the selected ISO image: Tiny Core, Alpine, Debian netinst or Ubuntu Server.
41. Allocate CPU based on CO2 AT1 sizing, but keep within safe host limits.
42. Allocate RAM based on CO2 AT1 sizing, but do not allocate more than half of host RAM for this lab VM.
43. Create a virtual disk. For Tiny Core or Alpine, 2-8 GB is enough for evidence. For Debian/Ubuntu, use 15-25 GB when storage permits.
44. Finish creation and start the VM.

VM Setting	Recommended Lab Range	Student Value
VM name	csa15-localvm-192511234	
OS image	Tiny Core Linux	

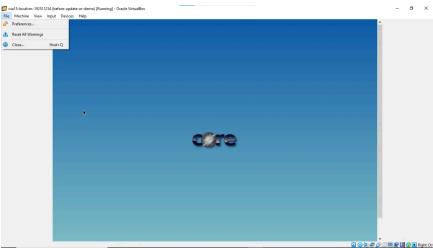
vCPU	Not captured from screenshot - to confirm in VM settings	
RAM	Not captured from screenshot - to confirm in VM settings	
Virtual disk	Not captured from screenshot - to confirm in VM settings	
Network mode	NAT (default; to confirm in VM network settings)	
Display / boot mode	Default	
Host Safety Guidance Do not allocate all host resources to the VM. Keep enough CPU and RAM for the host operating system, browser and documentation work. If the host has 8 GB RAM, a 1 GB or 2 GB VM is more appropriate than a 6 GB VM. Evidence Space: Paste VM settings screenshot before first boot 		

C. Boot and Verify the VM

45. Start the VM and wait for the OS boot screen.
46. For Tiny Core, confirm that the graphical desktop appears.
47. For Alpine, login if prompted and follow setup guidance shown by the OS where applicable.
48. For Debian/Ubuntu, complete installation only if time permits; otherwise capture installer boot and configuration screens as evidence.
49. Open terminal inside the VM where possible.
50. Run or record equivalent commands: `uname -a`, `free -h`, `df -h` and `ip addr`.

Checkpoint	Status	Evidence / Observation
VM starts without boot error.	☒ Yes ☐ Rework	Tiny Core boot logo displayed; VM running without boot error
OS screen or terminal is visible.	☒ Yes ☐ Rework	Tiny Core boot screen visible; terminal command verification still pending
CPU/RAM/disk allocation is verified.	☐ Yes ☒ Rework	Allocation not yet confirmed via terminal commands (<code>uname -a</code> , <code>free -h</code> , <code>df -h</code>) - pending

Network mode is recorded.	[X] Yes [] Rework	NAT mode (default) recorded
Evidence Space: Paste VM boot/login/terminal screenshots		



D. Snapshot and Clone Practice

- 51. Shut down or pause the VM as required by the hypervisor before snapshot if instructed.
- 52. Create a snapshot named before-update-or-demo.
- 53. Record snapshot name, date and reason.
- 54. Create a linked clone or full clone if the device has enough storage. If not, write the reason and create a clone plan.
- 55. Start the VM again and confirm it still boots.

Recovery Item	Student Evidence / Decision
Snapshot name	before-update-or-demo
Snapshot timing	Taken before update/demo session (visible in VM window title)
Snapshot reason	Rollback point in case an update or demo configuration change causes issues
Clone attempted?	No
Clone type	Not attempted
Storage impact observed	To be recorded if a clone is attempted
Evidence Space: Paste snapshot manager and clone wizard/evidence screenshot	

E. Local VM Security Reflection

Security Area	Student Decision
VM network mode	NAT

Why this mode is selected	NAT gives the VM internet access while isolating it from the local network - a safe default for lab use
Guest password policy	Not captured - to confirm
Shared folders enabled?	No
Clipboard/drag-drop enabled?	No
Risk if VM is exposed to public network	Tiny Core has minimal hardening by default; exposing it to a public network without firewall/authentication hardening would risk unauthorized access

Part 5: Track C - Docker Container Practical and Comparison

Docker Desktop: <https://www.docker.com/products/docker-desktop/>

This path helps students understand how a container differs from a VM. A VM virtualizes hardware and runs a guest OS. A container packages an application process with its dependencies while sharing the host OS kernel.

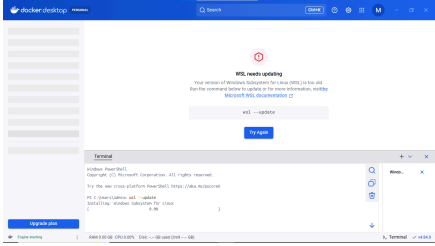
Dimension	Virtual Machine	Container
Isolation	Full guest OS isolation	Process-level isolation
Startup	Usually slower	Usually faster
Resource use	Higher due to guest OS	Lower and lightweight
Best fit	Full OS, database VM, legacy application, security boundary	API service, microservice, portable backend, quick deployment
Evidence in this AT	VM settings, boot, login, commands, snapshot/clone	Image, container run, port mapping, logs, service test

A. Container Verification Steps

56. Install Docker Desktop from the official site if allowed on the device.
57. Open Terminal or PowerShell and run: `docker --version`.
58. Run a lightweight container such as: `docker run --rm hello-world`.
59. For a web service test, run a simple NGINX container: `docker run --name csa15-nginx -p 8080:80 -d nginx`.
60. Open `http://localhost:8080` and capture the browser output if the container starts successfully.
61. Check logs using: `docker logs csa15-nginx`.
62. Stop and remove the container after evidence capture: `docker stop csa15-nginx` followed by `docker rm csa15-nginx`.

Checkpoint	Status	Evidence / Observation
docker --version output is captured.	☐ Yes ☒ Rework	Docker Desktop fails to start - 'Virtualization support not detected'; troubleshooting in progress (wsl --update running)
A container run command is captured.	☐ Yes ☒ Rework	Not yet - blocked by Docker Desktop startup failure
Port mapping or service evidence is captured if web test is used.	☐ Yes ☒ Rework	Not yet captured
Container logs or status are captured.	☐ Yes ☒ Rework	Not yet captured
Container is stopped/removed after evidence capture.	☐ Yes ☒ Rework	N/A - no container has been run yet

Docker Evidence	Student Observation
Docker version	Docker Desktop v4.84.0 installed; engine not yet started
Image used	Planned: hello-world, then nginx
Container name	Planned: csa15-nginx
Port mapping	Planned: 8080:80
Service URL tested	Planned: <code>http://localhost:8080</code>

Log/status observation	Docker Desktop currently blocked by virtualization detection error; fix in progress (WSL2 kernel update running via wsl --update)
Stop/remove evidence	Pending
Evidence Space: Paste Docker command and browser/service screenshots	

B. Capstone Module Decision

Capstone Module	VM / Container / Managed Service	Technical Reason
Frontend	Managed Service	Static hosting + CDN for the web UI is cheaper and faster than a dedicated VM
Backend/API	VM	Core API needs a persistent process, direct DB connectivity and steady compute for auth/session logic
Database	Managed Service	Reliability, automated backup and scaling are critical for CO-attainment records
File storage	Managed Service	Object storage handles learning-resource uploads without consuming VM disk
AI/Python module	Container	Recommendation engine is packaged as a portable, independently scalable microservice
Analytics/dashboard	Container	Aggregation/reporting jobs scale out independently from the core API
Notification/background job	Container	Lightweight event-driven jobs suit fast container startup

C. Optional Container and Orchestration Extension

Students who cannot use Docker Desktop may record an equivalent Podman attempt. Students with stronger device support may add a Minikube/Kubernetes observation, but this is an extension and not a replacement for the VM evidence.

Podman: <https://podman.io/>

Minikube: <https://minikube.sigs.k8s.io/docs/start/>

Kubernetes documentation: <https://kubernetes.io/docs/home/>

Optional Tool	Possible Evidence	Student Observation
Podman	Not attempted	
Minikube	Not attempted	
Kubernetes	Not attempted	

Part 6: Track D - Technical Simulation Path

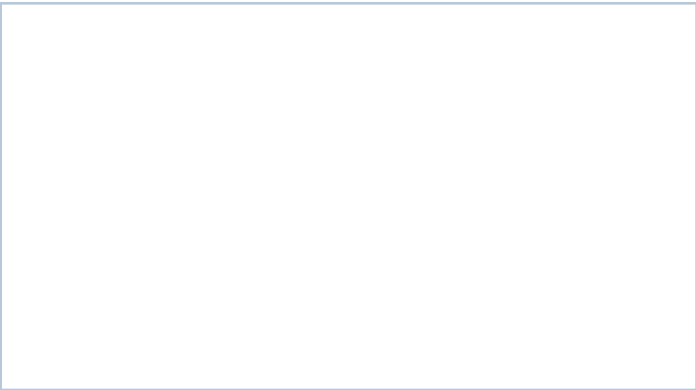
Use Only When Required

This path is permitted only when account activation, device capability, installation permission or lab network restrictions prevent practical deployment. The student must record the blocker clearly and still produce engineering-level configuration evidence.

CloudSim: <https://github.com/Cloudslab/cloudsim>

Simulation Artifact	Required Content
Blocker statement	Not applicable - Azure and Docker tracks are being completed practically; simulation path not used
VM configuration table	N/A
Security table	N/A
Snapshot/clone plan	N/A
Command plan	N/A
Architecture diagram	N/A
Risk analysis	N/A
Blocker Evidence and Justification	

Planned Configuration	Student Value
Platform	Azure / VirtualBox / VMware / Docker
OS image	N/A
vCPU	N/A
RAM	N/A
Disk	N/A
Network mode / firewall	N/A
Access method	N/A
Service to verify	N/A
Paste or draw the simulation architecture diagram here	



Part 7: CO2 Reflection and Infrastructure Defense

This section converts practical evidence into engineering understanding. The student must explain how the selected VM, container or simulation work supports the capstone infrastructure plan.

Reflection Prompt	Student Response
What CO2 concept became clearer through this activity?	The practical difference between VM-level virtualization (VirtualBox, Azure VM) and container-level isolation (Docker) - including OS boot behaviour, resource allocation and network modes
How did the practical configuration confirm or challenge the CO2 AT1 VM sizing estimate?	CO2 AT1 recommended 8 vCPU/32GB RAM/100GB SSD for the full production-scale AI workload; the lab evidence VMs (Tiny Core local VM, planned Azure B-series) are deliberately much smaller because they demonstrate configuration and security mechanics rather than full recommendation-engine inference capacity
Which evidence proves that CPU, RAM, disk, OS image and network settings were understood?	VM creation with named resources, Tiny Core boot verification, snapshot naming (before-update-or-demo), and planned terminal verification commands (uname -a, free -h, df -h, hostname -l) for the Azure VM
Which security decision is most important for the capstone product and why?	Restricting inbound ports to SSH only (22) and avoiding public exposure of services until explicitly required and time-boxed, since the capstone handles sensitive student CO-attainment records
What will be carried forward from CO2 AT2 to CO2 AT3 infrastructure defense?	The two-tier design (app VM + managed database), the containerized recommendation engine, snapshot/rollback discipline, and a minimal open-port security posture
Short Defense Statement: In 8-10 technical lines, defend whether the selected VM/container setup is suitable for the capstone prototype. The current setup is a reasonable early-stage validation, not yet the final capstone footprint. The local VirtualBox VM confirms basic boot, resource allocation and snapshot rollback discipline (before-update-or-demo), which mirrors how the production app VM will be protected before updates and demos. The planned Azure B-series VM validates cloud deployment mechanics (region, image, SSH-only access) on the exact platform chosen for the pilot, even though its size is intentionally far below the CO2 AT1 estimate of 8 vCPU/32GB, since this AT is about correctness of configuration, not full inference load. Docker is the right fit for the recommendation engine and analytics jobs because it isolates and scales the AI/Python workload independently of the core API, matching the CO2 AT1 VM/container decision table. The current Docker Desktop virtualization error is a host-configuration issue, not a design flaw, and is being resolved via a WSL2 kernel update. Overall, the VM/container mix is suitable for the prototype stage; it will need to move to the full two-tier design with a managed database once real user traffic is introduced.	

Student-Facing Assessment Criteria

Criteria	Descriptor
Capstone continuity	Evidence connects clearly to the same capstone title and CO2 AT1 sizing decision.
Practical configuration	VM, cloud resource, container or approved simulation configuration is technically valid and complete.
Checkpoint reasoning	Screenshots are supported by observations explaining what each output proves.
Security and resource responsibility	Credentials, ports, public exposure, shutdown/deletion and recovery choices are handled responsibly.
Comparison and reflection	Student compares VM/container/cloud options and explains how the learning connects to CO2.
Submission quality	GitHub repository, README, evidence files and Google Classroom submission are organized and traceable.

Part 8: Evidence Index, GitHub and Google Classroom Submission

The submission must be traceable. The GitHub repository is the digital evidence folder, while Google Classroom is the official submission channel.

Repository Item	Expected Content
README.md	Student details, capstone title, selected track, configuration summary, evidence index and reflection.
Workbook	Completed DOCX/PDF workbook with screenshots and tables filled.
Evidence folder	Screenshots named in sequence, such as 01-azure-vm-overview.png.
Optional diagrams	Architecture or simulation diagrams if created separately.
Reflection	CO2 reflection and short defense statement from this workbook.

Evidence No.	File / Screenshot Name	What It Proves
1	01-azure-account-activation.png	Confirms Azure for Students subscription is active with full credit balance
2	02-docker-desktop-error.png	Shows the initial Docker Desktop virtualization error and the troubleshooting starting point
3	03-localvm-tinycore-running.png	Shows the VirtualBox VM running Tiny Core Linux with the before-update-or-demo snapshot label visible
4	Pending	To be added once the Azure VM is deployed
5	Pending	To be added once terminal verification commands are captured

6	Pending	To be added once Docker container evidence is captured
7	Pending	To be added
8	Pending	To be added
Checkpoint	Status	Evidence / Observation
Repository is created under student's GitHub account.	☐ Yes ☒ Rework	Not yet created
README.md is complete.	☐ Yes ☒ Rework	Not yet written
Workbook is uploaded.	☐ Yes ☒ Rework	Not yet uploaded
Screenshots are uploaded and named clearly.	☐ Yes ☒ Rework	Not yet uploaded
Google Classroom submission contains the GitHub repository link.	☐ Yes ☒ Rework	Not yet submitted
No passwords, private keys, tokens or sensitive account details are uploaded.	☒ Yes ☐ Rework	Confirmed - no passwords, keys or tokens appear in any screenshot used in this workbook

Student Assurance

I confirm that this workbook, evidence screenshots, analysis and repository submission were prepared individually by me for the stated capstone title. I have not uploaded passwords, tokens, private keys or sensitive account information.

Student Name	Mukilan R
Register Number	192421043
Signature	
Date	

Faculty Verification

Faculty Name	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Constructive Feedback / Remarks	
Strength Observed	
Improvement Suggested	
Signature / Date	

Reference Links Used in This Workbook

Azure for Students: <https://azure.microsoft.com/en-us/free/students>

Azure for Students offer details: <https://azure.microsoft.com/en-us/pricing/offers/ms-azr-0170p>

Azure Linux VM quickstart: <https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal>

VirtualBox downloads: <https://www.virtualbox.org/wiki/Downloads>

VMware Workstation and Fusion: <https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion>

Tiny Core Linux downloads: <https://distro.ibiblio.org/tinycorelinux/downloads.html>

Alpine Linux downloads: <https://alpinelinux.org/downloads/>

Alpine virt ISO directory: https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/

Debian netinst: <https://www.debian.org/download>

Debian network install: <https://www.debian.org/CD/netinst/>

Ubuntu Server: <https://ubuntu.com/download/server>

Docker Desktop: <https://www.docker.com/products/docker-desktop/>

Podman: <https://podman.io/>

Minikube: <https://minikube.sigs.k8s.io/docs/start/>

Kubernetes: <https://kubernetes.io/docs/home/>

CloudSim: <https://github.com/Cloudslab/cloudsim>